CI-WORKFLOW-AUDIT

CI Workflow Audit — `.github/workflows/ci.yml`

> **The audited file is disabled (2026-08-27).** By operator decision — ***"Comply — disable both, enforce locally."*** — `.github/workflows/ci.yml` is renamed to a non-active `.disabled` name per §11.4.156(B), and the gates it ran move to a **local pre-push hook**. ⚠ **Scope correction, same day: only `ci.yml` was disabled.** The *"disable both"* decision was partially reversed — `milosvasic.ru/.github/workflows/pages.yml` is **ACTIVE and stays active**, because it is the sole publish path for a live production site. That does not affect this audit, whose subject is `ci.yml` alone; it affects only the cross-references in §6. **The path `.github/workflows/ci.yml` used throughout this document therefore no longer resolves**; read every reference to it as the `.disabled` file. The audit's *findings* are unaffected — they are about the accuracy of the file's own claims, and they transfer to the renamed file and to the hook that inherits its gate definitions. In particular **S2's Gate 2 vacuous-pass finding is now more load-bearing, not less**: it is about `_tools/audit-hardcoding.sh`, which the local hook will invoke, and it remains unfixed. See `../constitution-adoption/DECISION-11-4-156-COMPLY.md`.

Scope. Every comment, step name, and `run:` block in `.github/workflows/ci.yml`, verified against the actual repository contents.

Date: 2026-08-27 **Method:** read-only. Files read; `bash -n` / `node --check` on every script the workflow invokes; Gates 1, 2 and 3 executed locally (all three write only to `mktemp` dirs / the Go build cache, never to the working tree); Gates 4, 5 and 6 were **not** executed because they write evidence into `_tests/evidence/`. No file in the repository was modified by this audit except this report.

Line numbers refer to the 215-line revision of `ci.yml` present in the working tree at audit time (HEAD `62706a9` + the concurrent in-flight edit that rewrote the old lines 44–49 and inserted “Gate 0”). `ci.yml` is being edited concurrently; re-anchor before applying corrections.

Verdict counts

Verdict	Count
FALSE	3
STALE	5
UNVERIFIABLE	4
VERIFIED	34
Total claims checked	**46**

---

1. FALSE

Claims that are contradicted by the repository as it stands.

#	Line	Claim (quoted)	What is actually true	Suggested correction
F1	79	`# Playwright/ chromium is happier with an explicit browsers path we cache-key on.`	**Nothing caches on it.** There is no `actions/cache` step anywhere in the workflow, and the only cache in play is `cache: npm` on `actions/ setup-node@v4` (L112), keyed on `_tests/package-lock.json` (L113) — that caches the npm cache, not `$GITHUB_WORKSPACE/.pw-browsers`. `npx playwright install --with-deps chromium` (L165) re-downloads chromium on **every** run. `.pw-browsers` is also not in `.gitignore`.	Either drop the words “we cache-key on” (`# Explicit browsers path so the install location is deterministic.`), or actuall add an `actions/cache` step keyed on `${{ runner.os }} pw-${{ hashFiles('_tests/ package-lock.json') }}` w `path: $ {{ github.workspace }}/.p browsers`, placed before L164. If the dir is kept, add `.pw-browsers/` to `.gitignor`
F2	39–40	`` `_site/` is git-ignored in that submodule (only a ``	`milosvasic.ru/.gitignore:70` does ignore `_site/`, and 8 files are force-tracked — but **three of them are**	`…`_site/` is git-ignored that submodule; 8 files a force-tracked (assets, pl a stale `_site/index.html`

#	Line	Claim (quoted)	What is actually true	Suggested correction
		handful of asset files are force-tracked — NOT the rendered pages)	**rendered pages**, not assets: `_site/index.html`, `_site/feed.xml`, `_site/pages/Cover_Letter.md`. (Full list: `_site/assets/css/style.css`, `_site/assets/images/milosvasic.png`, three PDFs under `_site/assets/pdf/`, plus the three above.) The *conclusion* — that a `jekyll build` is required because the rest of the site is not tracked — still holds.	and `_site/feed.xml`) bu the localized page tree not. So we run `jekyll build` here…
F3	35–36	constitution/ and design-toolkit/ are governance/ design submodules that no gate command reads, so they are intentionally not fetched.	**There is no `constitution/` submodule.** `.gitmodules` registers it at `submodules/constitution`. The enumeration is also incomplete: the repo has **7** submodules — `ai_interviewing`, `design-toolkit`, `milosvasic.ru`, `monetization`, `submodules/constitution`, `submodules/superspec`, `vasic.digital` — so **five** are deliberately not fetched, not two. (The "no gate command reads them" part is correct: no gate script or spec references any of the five.)	The other five submodules ai_interviewing, design- toolkit, monetization, submodules/constitution, submodules/superspec — a governance/design/produc submodules that no gate command reads, so they a intentionally not fetche

2. STALE

Claims that were true once, or are true in outline but no longer describe the file or the tree accurately.

#	Line	Claim (quoted)	What is actually true	Suggested correction
S1	13–22	`Gates wired here (each is a separate step, fail-fast):` followed by a list numbered **1–6**	The workflow now has **seven** gates. `Gate 0 — hardcoded path audit` (`./scripts/audit-hardcoded-paths.sh`) was added at L131–140 and is **absent from the header list**. Every gate 0–6 resolves to exactly one real step; none duplicated, none missing on the step side — the header index is what is out of date.	Insert as the first list entry: `# 0. Hardcoded path audit ./scripts/audit-hardcoded-paths.sh` and keep the header list adjacent to any future gate insertion.
S2	9–10	`Every gate below is a REAL check that can genuinely fail — no vacuous always-pass steps.`	True for gates 0, 1, 3, 4, 5 (all mutation-paired or asserting on real content). **Gate 2 has a vacuous-pass path**: `_tools/audit-hardcoding.sh` uses `set -uo pipefail` *without* `-e` and runs the generator as `"$BIN" -site … >/dev/null 2>&1` in a loop whose exit status is never checked. If the generator produced nothing, all four checks would grep an empty `$OUT` and report , exiting 0. (It does currently produce output and does currently pass — verified by running it.)	Either soften to `Every gate below is a REAL check that can genuinely fail`, or — better — fix the script: check the generator's exit status inside the loop, and assert `$OUT` is non-empty before the four content checks. Then the

#	Line	Claim (quoted)	What is actually true	Suggested correction
				header claim becomes fully true.
S3	188–189	`it is the spec designed to run LIVE against production (VD_BASE/MV_BASE) and is far too heavy for per-push CI`	The spec's own header says `Default: runs against the locally-served build (playwright webServer on :8401 / :8082)`, and its inline comment says `Local build resolves in seconds; a LIVE GitHub-Pages crawl is thousands of network round-trips` — hence `test.setTimeout(isLive ? 1_800_000 : 600_000)`. CI sets neither `VD_BASE` nor `MV_BASE`, so it would run **locally, in seconds**, inside a 45-minute job. The exclusion may still be desirable (it is the slowest local spec), but "far too heavy for per-push CI" is contradicted by the spec.	`…it is the spec designed to run LIVE against production (VD_BASE/MV_BASE); we exclude it here so the local run stays a smoke of the shipped pages rather than a full sitemap crawl.` Or drop the exclusion — the honest reading is that it would pass locally.
S4	181–183	`# (playwright.config.js now resolves its static roots relative to the repo # via __dirname, so the former /Volumes symlink bridge is no longer needed — # the suite runs from any`	Factually correct (`_tests/playwright.config.js:7` — `const REPO = path.resolve(__dirname, '..')`), but it is now a **duplicate** of the rewritten header block at L44–56, which says the same thing at greater length. Two copies of the same fact in	Delete L181–183. The header block L44–56 is the single place this belongs.

#	Line	Claim (quoted)	What is actually true	Suggested correction
		`checkout location, including a fresh clone.)`	one file are exactly how the previous `/Volumes` comment went stale.	
S5	22	`(see the note on step 6 below)`	There is no step called “step 6”. The block below is labelled `# ---- Gate 6 ----` and the step is named `Gate — Playwright (chromium), excluding the live all-language crawl.` Cosmetic, but it is the kind of drift that makes a reader hunt.	`(see the note on Gate 6 below)`

3. UNVERIFIABLE

Not checkable from a read-only offline audit of this checkout. Listed so the audit is honest about its own limits, not as defects.

#	Line	Claim	Why unverifiable	How to verify
U1	32–33, 95	`(both are public GitHub Pages repos)` / `Both repos are public.`	No network access in this audit. The two HTTPS URLs at L97–98 do match the SSH URLs in `.gitmodules`, and both pinned commits (`6e5411c` vasic.digital, `66c8d60` milosvasic.ru) are contained in local `origin/main` /	`git clone --depth 1 https://github.com/vasic-digital/vasic-digital.github.io.git` from an unauthenticated shell.

#	Line	Claim	Why unverifiable	How to verify
			`github/main`, so the objects exist upstream — but anonymous readability is not provable offline.	
U2	10–11	`Each was proven to pass locally before this file was committed (see the PR description / task evidence block).`	No PR description or "task evidence block" exists anywhere in the tree; the pointer dangles.	Either link a concrete artifact under `_tests/evidence/` or drop the parenthetical. Note: gates 1, 2 and 3 **were** re-proven to pass during this audit (see §5).
U3	193–195	Whether the full chromium Playwright suite (21 spec files, minus `all-language`) passes on a runner	Executing it writes into `_tests/evidence/` and `_tests/test-results/`, which this audit was forbidden to do. Structurally it is sound (see §5 for the specific risk list).	Run `npx playwright test --project=chromium --grep-invert "all-language"` on a throwaway clone.
U4	129	`sudo apt-get install -y --no-install-recommends poppler-utils tesseract-ocr` provides working OCR	On Ubuntu, `tesseract-ocr` **Depends** on `tesseract-ocr-eng` + `tesseract-ocr-osd`, so `--no-install-`	`apt-cache depends tesseract-ocr` on `ubuntu-latest`. Low risk; note that `_tests/export/validate-pdf.js` treats a missing binary as SKIP-with-reason, but

#	Line	Claim	Why unverifiable	How to verify
			`recommends` should still yield English OCR — but this was not confirmed against the runner image's package graph. If `eng` were only *Recommended*, `tesseract` would run and produce empty text.	an *installed-yet-langless* tesseract would produce a real FAIL, not a SKIP.

4. VERIFIED

Explicitly checked and correct. Listed so this audit demonstrates coverage rather than only complaints.

4.1 Referenced files exist

#	Line	Claim	Evidence
V1	14, 143–145	`cd _tools/gen && go test ./...`	`_tools/gen/go.mod` exists; 15 `.go` files, **6** `*_test.go` (`data`, `home`, `markdown`, `portfolio`, `product`, `seo`). Not a vacuous package.
V2	15, 149	`bash _tools/audit-hardcoding.sh`	Exists, executable, `bash -n` clean.
V3	16, 153	`bash _tools/translate/`	Exists, `bash -n` clean.

#	Line	Claim	Evidence
		`reproducibility-selftest.sh`	
V4	17, 157	`bash _tools/portfolio/self-validate.sh`	Exists, `bash -n` clean; its `validate.mjs`, `selfvalidate-fixtures/good.json` and `bad.json` all exist.
V5	18, 172	`bash _tests/run-harness-selfvalidation.sh`	Exists; delegates to `_tests/visual/self-validate.sh` and `_tests/export/self-validate.sh`, both of which exist and are `bash -n` clean.
V6	113	`cache-dependency-path: _tests/package-lock.json`	Exists, `lockfileVersion: 3`, and **in sync** with `_tests/package.json` (`@playwright/test@1.61.0`, `pixelmatch@7.2.0`, `pngjs@7.0.0`, `@axe-core/playwright@4.11.3` + transitives). `npm ci` will not reject it.
V7	100–101	`ls vasic.digital/index.html` / `ls milosvasic.ru/Gemfile`	Both exist. Both are the correct existence probes for the two submodules.
V8	140	`./scripts/audit-hardcoded-paths.sh`	Exists, executable, `bash -n` clean. `.hardcoded-paths-allow` exists and allow-lists exactly one file (the detector itself, with a stated reason).
V9	176–177		`milosvasic.ru/Gemfile` pins `jekyll`

#	Line	Claim	Evidence
		`bundle exec jekyll build` in `milosvasic.ru`	`~> 4.4` + `jekyll-seo-tag`, `jekyll-feed`; `Gemfile.lock` present (`BUNDLED WITH 2.5.11`) with `x86_64-linux` in PLATFORMS — `bundler-cache: true` will resolve on the runner.
V10	203, 212	`path: _tests/evidence/**` / `path: _tests/test-results/**`	`_tests/evidence/` exists; `playwright.config.js` writes its HTML report to `evidence/html-report` and (default `outputDir`) traces/screenshots to `_tests/test-results`. Both artifact paths resolve.

4.2 Scripts do what the comments say

#	Line	Claim	Evidence
V11	15, 148	`(builds the Go generator)`	`_tools/audit-hardcoding.sh` runs `go build -ldflags "-X main.buildYear=2099" -o "$BIN" .` in `_tools/gen`.
V12	16	`Reproducibility guard`	`reproducibility-selftest.sh` is a pure static scan of `${HELIX_BIN:-…}` / `${HELIX_TRANSLATE_BIN:-…}` defaults in `_tools/**/*.sh` and `ENGINE=`/`HELIX_BIN=` in `_tools/**/*.py`, failing on any `/tmp/` default. It derives `REPO` from `$SELF_DIR/../..`; no network, no engine

#	Line	Claim	Evidence
			needed. Description matches.
V13	17, 156	`portfolio §1.1 data-integrity self-validation`	`self-validate.sh` is genuinely mutation-paired: golden-good must exit 0, golden-bad must exit non-zero, and it prints `GATE: PORTFOLIO-DATA-INTEGRITY (§1.1)`. It needs only bare `node` (`node:fs`) — correctly placed **before** `npm ci`.
V14	19–20, 169–170	`visual §11.4.170 via headless chromium + export §11.4.168 via poppler + tesseract OCR`	`_tests/visual/visual-oracle.js` uses `require('@playwright/test').chromium` (satisfied by `npm ci`, browser by `playwright install`); `_tests/export/validate-pdf.js` shells out to `pdftotext`, `pdfimages`, `pdftoppm`, `tesseract`. The §-numbers printed by both scripts match the ones in the step name.
V15	127–128	`The export validator SKIPs with a reason if a tool is missing, but we install them so the gate runs for real rather than degrading to SKIP.`	`validate-pdf.js:43 hasTool()` + `:85/:114/:125/:127` emit `SKIP` with an explicit reason string ("…missing — cannot…", "SKIP-with-reason, not a PASS"). Exactly as described.
V16	170	`Fixtures are committed, so nothing is rebuilt.`	**Load-bearing and true.** `_tests/export/fixtures/golden-good.pdf` and `golden-bad.pdf` are tracked (`git ls-files`), so `export/self-validate.sh`'s rebuild branch never fires. This matters: `build-fixtures.sh` needs `pandoc`

#	Line	Claim	Evidence
			+ `weasyprint`, which the workflow does **not** install. Visual fixtures (`good.html`, `bad.html`) are tracked too.
V17	159	`# ---- Node deps + browser (needed by gates 5 and 6) ----`	Correct and correctly placed. Gates 0–4 need no npm packages (Gate 4 uses only Node builtins); Gate 5's visual oracle needs `@playwright/test` + a chromium binary; Gate 6 needs both.
V18	133–136	`This repo previously carried 33 such paths across 18 files … one of them made _tools/deploy-langs.sh cd into a nonexistent directory SILENTLY (it sets -uo pipefail but not -e) and then commit and push both site submodules.`	Matches commit `72dc135` *"Remove all hardcoded machine paths (18 files, 33 occurrences) + audit guard"*. `git show 72dc135^:_tools/deploy-langs.sh` line 7 is `set -uo pipefail`, line 8 is `ROOT="<macos-host>/Projects/vasic"`; today's line 14 derives `ROOT` from `${BASH_SOURCE[0]}`. The script really does `git -C "$s" commit` (:96) and `git -C "$s" push "$r" main` (:100).
V19	137–138	`Comment-only mentions are ignored, $HOME/~ are fine, and genuine exceptions live in .hardcoded-paths-allow.`	`audit-hardcoded-paths.sh` `strip_comments()` filters `#` / `//` / `*` / `/*` leading lines per extension; `PATTERN` deliberately omits `$HOME`/`~`; `is_allowed()` reads `.hardcoded-paths-allow`. All three sub-claims hold.
V20	49–52	`_tests/playwright.config.js now derives its static roots itself with`	`playwright.config.js:7-9`: `const REPO = path.resolve(__dirname, '..')`, `VD_ROOT = path.join(REPO, 'vasic.digital')`,

#	Line	Claim	Evidence
		`path.resolve(__dirname, '..')`	`MV_ROOT = path.join(REPO, 'milosvasic.ru', '_site')`. No absolute path in the file.

4.3 Toolchain versions

#	Line	Claim	Evidence
V21	103, 106	`Set up Go (matches _tools/gen/go.mod → go 1.26)` / `go-version: '1.26'`	`_tools/gen/go.mod` reads exactly `go 1.26`. Comment and input agree. Verified locally against `go1.26.2`.
V22	108, 111	`Set up Node 20` / `node-version: '20'`	Consistent. Every locked package's `engines` is satisfied: `@playwright/test`, `playwright`, `playwright-core` → `node >=18`; `pngjs` → `>=14.19.0`; `axe-core` → `>=4`. `_tests/package.json` declares no `engines` constraint that Node 20 violates.
V23	115, 118	`Set up Ruby + Jekyll` / `ruby-version: '3.3'`	Matches the sibling deploy workflow `milosvasic.ru/.github/workflows/pages.yml`, which also pins `3.3`. `Gemfile` has no conflicting `ruby` directive.
V24	126, 129	`poppler-utils -> pdftotext/pdfimages/pdftoppm ; tesseract-ocr -> OCR.`	Exactly the four binaries `validate-pdf.js` probes (`:72`) — `pdftotext`, `pdfimages`, `pdftoppm` from poppler-utils, `tesseract` from

#	Line	Claim	Evidence
			tesseract-ocr. Package→binary mapping is correct. `visual-oracle.js:211` uses `tesseract` too, guarded by `hasTool` and wrapped in try/catch (OCR is a soft signal there, not a hard assertion).

4.4 Gate numbering / naming consistency

#	Line	Claim	Evidence
V25	131–195	Gate markers resolve to real steps	`Gate 0` → `Gate — hardcoded path audit` (L139); `Gate 1` → `Gate — Go unit tests (_tools/gen)` (L143); `Gate 2` → `Gate — hardcoding audit` (L148); `Gate 3` → `Gate — HelixTranslate reproducibility self-test` (L152); `Gate 4` → `Gate — portfolio §1.1 …` (L156); `Gate 5` → `Gate — harness self-validation …` (L171); `Gate 6` → `Gate — Playwright (chromium) …` (L193). **Seven markers, seven steps, no duplicates, no**

#	Line	Claim	Evidence
			dangling markers. The only inconsistency is the header list omitting Gate 0 (→ S1).
V26	82	`Checkout (no auto-submodules — see deviation (a) in the header)`	Deviation (a) exists at L26–36 and does explain `submodules: false` (L85). Cross-reference resolves.

4.5 Paths / machine / environment assumptions

#	Line	Claim	Evidence
V27	96	`git submodule init vasic.digital milosvasic.ru`	Both are real submodule **paths** in `.gitmodules` (`path = milosvasic.ru`, `path = vasic.digital`), so `submodule init <path>` addresses them correctly.
V28	90–93, 97–98	`The gitlinks are registered with SSH URLs (git@github.com:...) which CI has no key for` + the two HTTPS overrides	`.gitmodules` registers `git@github.com:vasic-digital/vasic-digital.github.io.git` and `git@github.com:milos85vasic/milosvasic.ru.git`. The HTTPS URLs at L97–98 are the exact same repos over `https://`. `git config submodule.<name>.url` is the right override key and takes effect for `git submodule update`.
V29	26–29	`A recursive checkout of the umbrella is KNOWN to fail: milosvasic.ru embeds a`	Substantively correct. `milosvasic.ru/.gitmodules` registers `Upstreamable` → `git@github.com:red-elf/`

#	Line	Claim	Evidence
			`Upstreamable.git`; `git ls-tree HEAD` in `milosvasic.ru` shows `160000 commit 94f9831… Upstreamable`. Inside it, `Upstreamable/.gitmodules` is **0 bytes** while `git ls-tree HEAD` still lists `160000 commit 3da1755… Upstreamable` — a gitlink with no URL, which is precisely what breaks `--recursive`. *Wording nit only:* a `.gitmodules` file is not itself "a gitlink"; suggest `…whose own .gitmodules is 0 bytes and therefore cannot resolve its nested Upstreamable gitlink.`
V30	29–30	`This is the exact reason milosvasic.ru/.github/workflows/pages.yml sets submodules: false.`	`pages.yml` header: *"The Upstreamable submodule has a broken nested gitlink (Upstreamable/Upstreamable with no .gitmodules URL) that causes the default recursive checkout to fail"*, and `submodules: false` with an inline comment saying the same. Verified.
V31	33–35	`Upstreamable is excluded from the Jekyll build anyway (milosvasic.ru/_config.yml exclude:), so nothing is lost.`	`milosvasic.ru/_config.yml` `exclude:` lists `Upstreamable` and `Upstreams`. Verified.
V32	41–42	`vasic.digital, by contrast, is committed static HTML and is served directly — no build.`	`vasic.digital/` contains committed `index.html`, `sitemap.xml`, `robots.txt` and per-language dirs (`ar be de es fa fr hi ja kk ko ru sr tr zh`) plus `products/`, `portfolio/`, `articles/`. Its `.gitignore` ignores only secrets, `node_modules/`, OS

#	Line	Claim	Evidence
			cruft — no HTML. `playwright.config.js` serves `VD_ROOT` directly with no build step.
V33	174, 186	`Build the Jekyll site Playwright will serve on :8082` / `The config auto-starts a python http.server for BOTH sites.`	`playwright.config.js` `webServer` array: `python3 -m http.server 8401 --directory ${VD_ROOT}` and `python3 -m http.server 8082 --directory ${MV_ROOT}` where `MV_ROOT = milosvasic.ru/_site`. Port and build target both correct. `python3` is present on `ubuntu-latest`.
V34	187–192	`The title is "... exhaustive all-language link & sitemap integrity", so the substring "all-language" (no trailing s) reliably excludes both its per-site variants and nothing else.`	`_tests/tests/all-languages-link-integrity.spec.js:55` — `test(`${site.key} — exhaustive all-language link & sitemap integrity`, …)` inside `for (const site of SITES)` with `SITES = vasic.digital + milosvasic.ru` → exactly two variants. `grep -rn "all-language" _tests/tests/` matches **only** that file. `VD_BASE` / `MV_BASE` are the real env vars (`:28`, `:33`). Correct on every particular, including the "no trailing s" detail (the file *name* is `all-languages-…`, the *title* is `all-language …`; the substring covers both).

Additional environment checks that produced no finding:

- **No machine-specific absolute path reaches any executed code.** `grep -rE "/Volumes|/Users/|/home/[a-z]"` over `_tests/tests/`, `_tools/`, `scripts/` and the live configs returns nothing. The remaining `<macos-host>/...` occurrences live in (a) two **untracked** `*.bak.20260827-180538` files and (b) `_tests/evidence/**` generated

artifacts — none tracked, none executed, and `_tests/evidence/` is in the Gate 0 `SKIP` list by design.
- **`_tests/evidence/html-report/`, `_tests/test-results/` and `_tests/node_modules/` are gitignored** at the umbrella root, with an explicit `!_tests/evidence/` negation keeping the curated evidence tree tracked. A fresh clone starts clean and CI's writes stay untracked.
- **Submodule pins are reachable upstream.** `git submodule status` is clean (no `+`/`-` prefixes); `vasic.digital@6e5411c` and `milosvasic.ru@66c8d60` are both contained in local `origin/main` / `github/main`, so `git submodule update` on a runner will find the commits.
- **`_tests/visual-effects.spec.js` is not collected.** It lives at `_tests/` root while `testDir: './tests'`, so the old `visual-effects.config.js` is not exercised by Gate 6.
- **This report's own path is safe for Gate 0.** `audit-hardcoded-paths.sh` skips `^docs/`, so quoting `<macos-host>/Projects/vasic` above cannot trip the gate it documents.
- **Header L4–6** — *"the ONLY GitHub Actions in play was milosvasic.ru/.github/workflows/pages.yml"* — holds as written. Two other workflow files exist in the tree (`.specify/extensions/superspec/.github/workflows/ci.yml`, tracked; `submodules/superspec/.github/workflows/ci.yml`, behind a gitlink), but GitHub executes only root `.github/workflows/`, so neither is "in play". `Constitution.md` reaches the same conclusion for superspec (third-party, out of §11.4.156(C) scope).

---

5. Would the workflow succeed on a fresh clone?

Gates 0–4: yes — three of them re-proven during this audit, two structurally sound.

Gate	Status	Evidence
0 — hardcoded path audit	**PASS (executed)**	`no machine-specific hardcoded paths (1 file(s) explicitly allowed)`
1 — Go unit tests	**PASS (executed)**	`ok vasic.digital/tools/gen 0.019s` under `go1.26.2`
2 — hardcoding audit	**PASS (executed)**	

Gate	Status	Evidence
		All four checks , `[audit] PASS — no hardcoded-content violations`
3 — reproducibility self-test	**PASS (executed)**	Both engine defaults repo-relative; no `/tmp` default in any `_tools/**/*.py`
4 — portfolio §1.1	**Structurally sound (not executed)**	Fixtures + validator present; needs only bare `node`, available from L108
5 — harness self-validation	**Structurally sound (not executed)**	Fixtures committed so the pandoc/weasyprint path never fires; chromium + poppler + tesseract all installed by L164/L129 before it runs at L171
6 — Playwright chromium	**Not verifiable offline**	See risks below

Step ordering is correct. Every dependency is installed before its consumer: submodules (L87) → Go (L103) → Node (L108) → Ruby (L115, `working-directory: milosvasic.ru`, which requires the submodule fetch to have already run — it has) → apt tools (L122) → gates 0–4 → `npm ci` (L160) → `playwright install` (L164) → Gate 5 (L171) → `jekyll build` (L175) → Gate 6 (L193). No step consumes something not yet present.

Residual risks for Gate 6 (unverified, ordered by likelihood):

1. `perf-budget.spec.js` asserts `maxBytes: 1_500_000` / `maxRequests: 45` against four pages including `/products/helixtrack.html` on both sites — those pages exist in both source trees, but byte budgets on a shared runner are the most plausible source of a first-run failure.
2. `retries: 1` is set globally, which absorbs timing flake but means a genuine failure costs double the wall time against `timeout-minutes: 45`.

3. `security-hardening.spec.js` and the a11y specs assert on shipped markup only (no server headers), so `python3 -m http.server` is an adequate host — but they were not run.
4. `--with-deps` requires `sudo` on the runner; fine on `ubuntu-latest`, would fail on a self-hosted runner without passwordless sudo.
5. U4 above: an installed-but-langless `tesseract` would turn Gate 5's export validator from SKIP into a hard FAIL.

Nothing found in this audit would block a fresh clone from running. The three FALSE items are all documentation defects, not execution defects: F1 costs a chromium re-download per run (no failure), F2 and F3 mis-describe the tree without changing behaviour.

---

6. Out of scope — noted, ~~not resolved~~ resolved 2026-08-27

As written at audit time:

> `.github/` at this repo root sits inside an unresolved governance conflict: `Constitution.md` §11.4.156(A) forbids an active CI workflow at a governed repo root, recorded as **OC-1** (`Constitution.md` §"OC-1 — Active CI at the repository root contradicts §11.4.156(A)"), with **OC-2** covering the same clause for `milosvasic.ru/.github/workflows/pages.yml` as an owned submodule, and gate `G4 — active CI contradicts §11.4.156` still marked **OPEN — operator decision**. This audit takes no position and changes nothing about it; every finding above is about the *accuracy* of the file's own claims, and applies whether or not the workflow is ultimately disabled per §11.4.156(B).

Update. The operator decision has been made: ***"Comply — disable both, enforce locally."*** `G4` is no longer *"OPEN — operator decision"*; it is **PARTIAL**. ~~Both `ci.yml` and `pages.yml` are renamed to a non-active `.disabled` name per §11.4.156(B)~~ — **`ci.yml` is**, and the gates this document audits move to a **local pre-push hook**. The audit's own closing caveat holds exactly as written: its findings are about the *accuracy* of the file's claims and apply whether or not the workflow is disabled — so **the three FALSE items and S1/S2 still need fixing**, now in the renamed file and in the hook that inherits its gate definitions.

Second update, same day — OC-2 went the other way. `milosvasic.ru/.github/workflows/pages.yml` was **not** disabled and will not be. `gh api repos/milos85vasic/milosvasic.ru/pages` returns

`build_type: "workflow"`, so that workflow is the **sole** publish path for the live production site (no `gh-pages` branch, no `docs/` folder, Jekyll source at the root; `_tools/deploy-langs.sh` pushes source and waits for that workflow rather than replacing it). Operator's overriding directive, verbatim: ***"Make sure all pages websites work flawlessly! No website can be broken! All websites we have here are running deployed in production!"*** **OC-2 therefore stays OPEN as a known, documented deviation — not an override.** A third surface, `vasic.digital`, is non-compliant at the *provider* level (`build_type: "legacy"`, `pages build and deployment` on every push) with **no file-level remedy**, since it has no workflow file at all.

Note that this makes **V30** and the **Header L4–6** verification in §4 read correctly again for the present tense: `pages.yml` still exists, still sets `submodules: false`, and is still the only GitHub Actions workflow in play across the site submodules.

An `Override §11.4.156` — the alternative this section anticipated — was sought and found not to exist: §11.4.156 refuses the exemption vocabulary by name, and a project carrier may only extend inherited rules, never weaken them. The OC-2 deviation is **not** an override either, and must not be written up as one. **Honest boundary (§11.4.6):** renaming `ci.yml` stops FILE-triggered runs only; org-default required workflows, branch-protection required checks and provider-side scheduled exports remain operator-only to change. They are no longer *unverified*, though — their state is measured on demand by `bash scripts/verify-provider-ci.sh` (0 = none found, 1 = confirmed, 2 = could not determine, which is **not** a pass), which the setup wizard runs as Step 9. Treat every provider fact in this report as a dated observation and re-measure. Full record: `../constitution-adoption/DECISION-11-4-156-COMPLY.md` §0.

7. Concurrent-edit note

Two items in this report describe content added by the in-flight edit that was already known and intentional, and are **not** raised as defects: the rewritten `/Volumes` header block (L44–56) and the new `Gate 0` step (L131–140). They were still audited on their merits — L44–56 verifies clean (V20), and Gate 0's factual claims verify clean (V18, V19) — but Gate 0's insertion is what left the header gate list at L13–22 incomplete (**S1**), so that one correction should land with the same edit.